

Objects and Classes

Instance Variables, and Methods

Instance variables belong to a specific instance.

Instance methods are invoked by an instance of the class.

Static Variables, Constants, and Methods

Static variables are shared by all the instances of the class.

Static methods are not tied to a specific object.

Static constants are final variables shared by all the instances of the class.

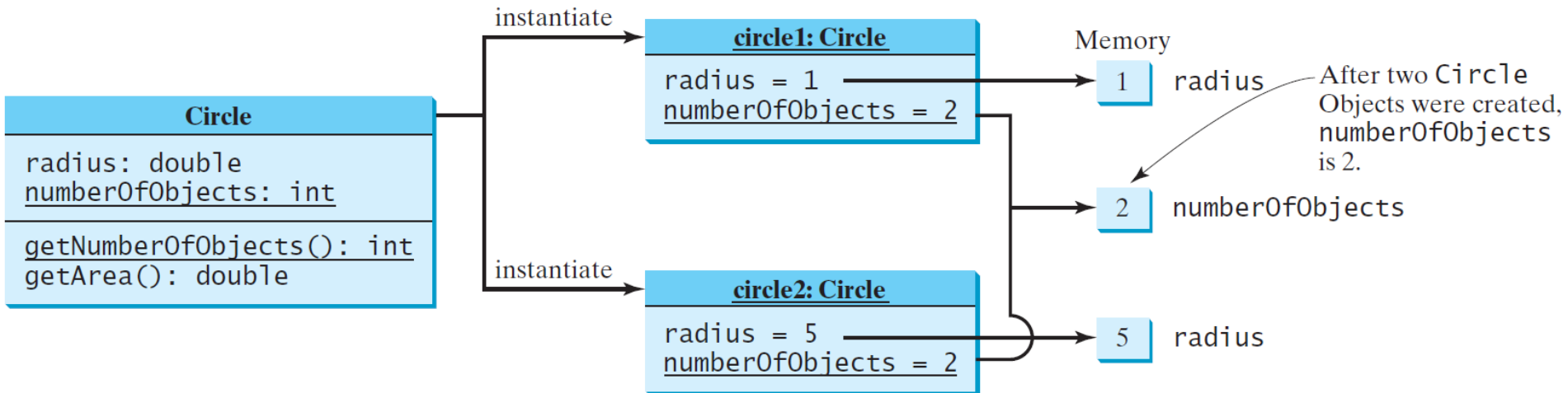
Static Variables, Constants, and Methods, cont.

To declare static variables, constants, and methods,
use the static modifier.

Static Variables, Constants, and Methods, cont.

UML Notation:

underline: static variables or methods



Example of Using Instance and Class Variables and Method

Objective: Demonstrate the roles of instance and class variables and their uses. This example adds a class variable `numberOfObjects` to track the number of `Circle` objects created.

`CircleWithStaticMembers`

`TestCircleWithStaticMembers`

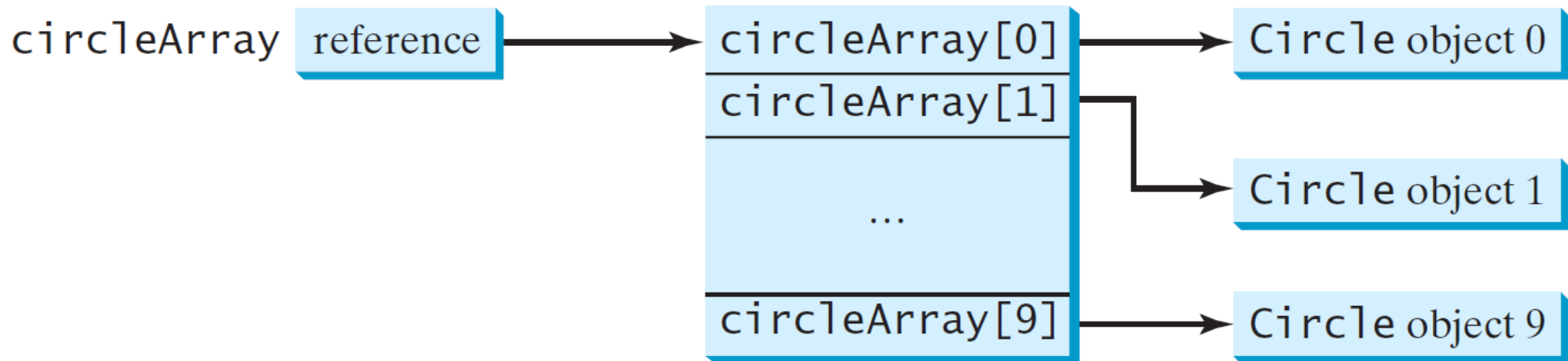
Array of Objects

```
Circle[] circleArray = new Circle[10];
```

An array of objects is actually an *array of reference variables*. So invoking `circleArray[1].getArea()` involves two levels of referencing as shown in the next figure. `circleArray` references to the entire array. `circleArray[1]` references to a `Circle` object.

Array of Objects, cont.

```
Circle[] circleArray = new Circle[10];
```



Array of Objects, cont.

Summarizing the areas of the circles

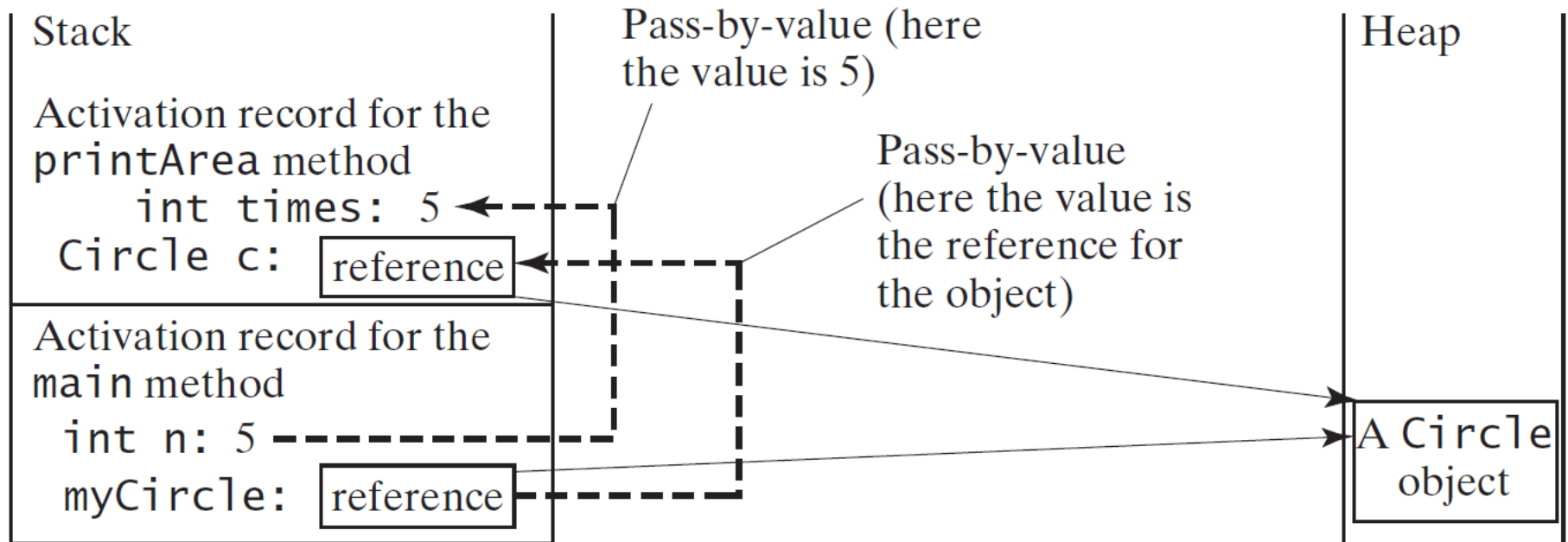
TotalArea

Passing Objects to Methods

- ❑ Passing by value for primitive type value
(the value is passed to the parameter)
- ❑ Passing by value for reference type value
(the value is the reference to the object)

TestPassObject

Passing Objects to Methods, cont.



Immutable Objects and Classes

If the contents of an object cannot be changed once the object is created, the object is called an *immutable object* and its class is called an *immutable class*. If you delete the set method in the Circle class, the class would be immutable because radius is private and cannot be changed without a set method.

A class with all private data fields and without mutators is not necessarily immutable. For example, the following class Student has all private data fields and no mutators, but it is mutable.

Example

```
public class Student {
    private int id;
    private BirthDate birthDate;

    public Student(int ssn,
        int year, int month, int day) {
        id = ssn;
        birthDate = new BirthDate(year, month, day);
    }

    public int getId() {
        return id;
    }

    public BirthDate getBirthDate() {
        return birthDate;
    }
}
```

```
public class BirthDate {
    private int year;
    private int month;
    private int day;

    public BirthDate(int newYear,
        int newMonth, int newDay) {
        year = newYear;
        month = newMonth;
        day = newDay;
    }

    public void setYear(int newYear) {
        year = newYear;
    }
}
```

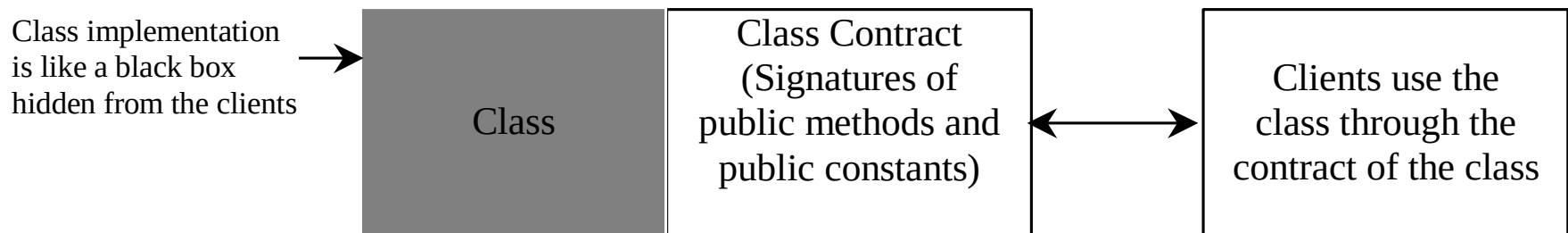
```
public class Test {
    public static void main(String[] args) {
        Student student = new Student(111223333, 1970, 5, 3);
        BirthDate date = student.getBirthDate();
        date.setYear(2010); // Now the student birth year is changed!
    }
}
```

What Class is Immutable?

For a class to be immutable, it must mark all data fields private and provide no mutator methods and no accessor methods that would return a reference to a mutable data field object.

Class Abstraction and Encapsulation

Class abstraction means to separate class implementation from the use of the class. The creator of the class provides a description of the class and let the user know how the class can be used. The user of the class does not need to know how the class is implemented. The detail of implementation is encapsulated and hidden from the user.



Designing the Loan Class

The UML serves as the contract for the class. A class user can use the class without knowing how the class is implemented.

Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, years, and loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

TestLoanClass

Loan

Designing the Loan Class

Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, year loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

From a class developer's perspective, a class is designed for use by many different customers. In order to be useful in a wide range of applications, a class should provide a variety of ways for customization through constructors, properties, and methods.

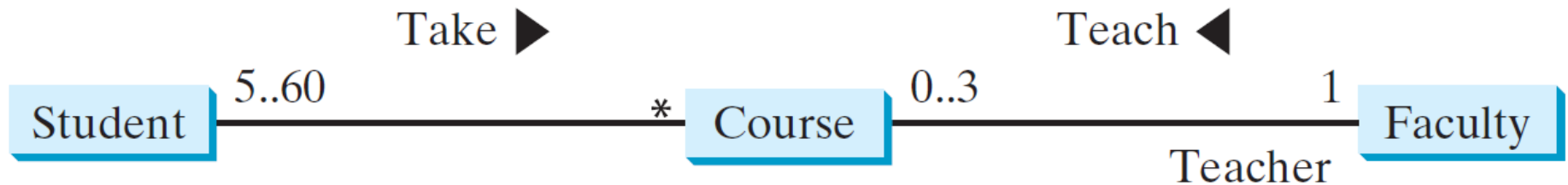
TestLoanClass

Loan

Class Relationships -Association

Association: is a general binary relationship that describes an activity between two classes objects.

Association manages: one-to-one, one-to-many, and many-to-many relationships.



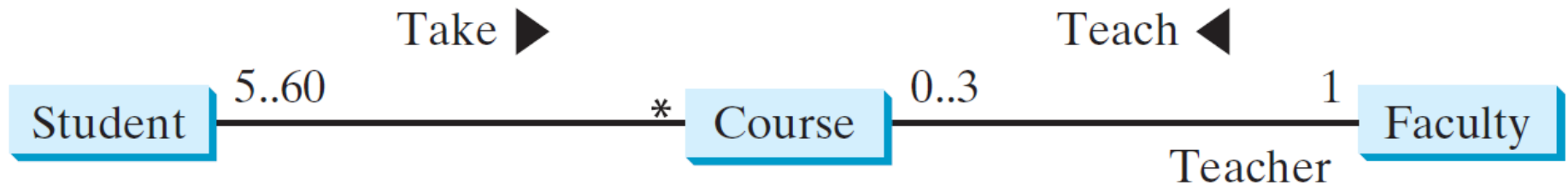
Class Relationships -Association

- In Java can implement associations by using data fields and methods.

```
public class Student {  
    private Course[]  
        courseList;  
  
    public void addCourse(  
        Course c) { ... }  
}
```

```
public class Course {  
    private Student[]  
        classList;  
    private Faculty faculty;  
  
    public void addStudent(  
        Student s) { ... }  
  
    public void setFaculty(  
        Faculty faculty) { ... }  
}
```

```
public class Faculty {  
    private Course[]  
        courseList;  
  
    public void addCourse(  
        Course c) { ... }  
}
```



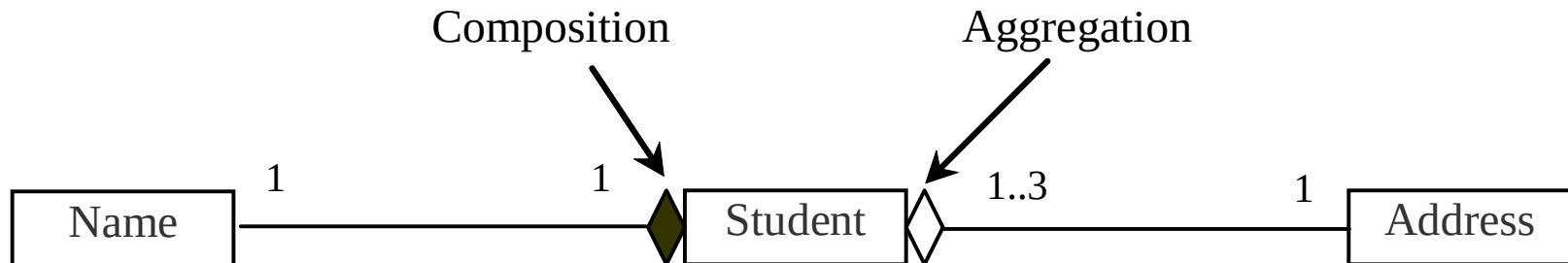
Class Relationships -Association

There are special types of associations:

- Aggregation
- Composition
- Inheritance

Aggregation and Composition

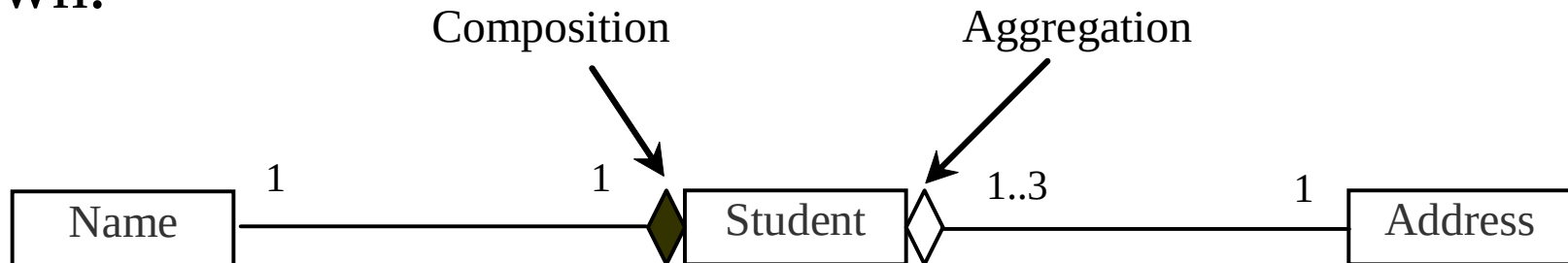
Aggregation models *has-a* relationships and represents an ownership relationship between two objects. The owner object is called an *aggregating object* and its class an *aggregating class*. The subject object is called an *aggregated object* and its class an *aggregated class*.



Aggregation and Composition

Composition is a special case of the aggregation relationship.

We refer aggregation between two objects as composition if the existence of the aggregated object is dependent on the aggregating object. In other words, if a relationship is composition, the aggregated object cannot exist on its own.



Aggregation and Composition

Since aggregation and composition relationships are represented using classes in similar ways, many texts don't differentiate them and call both compositions.

Aggregation and Composition

An aggregation and composition are usually represented as a data field in the aggregating class.

```
public class Name {  
    ...  
}
```

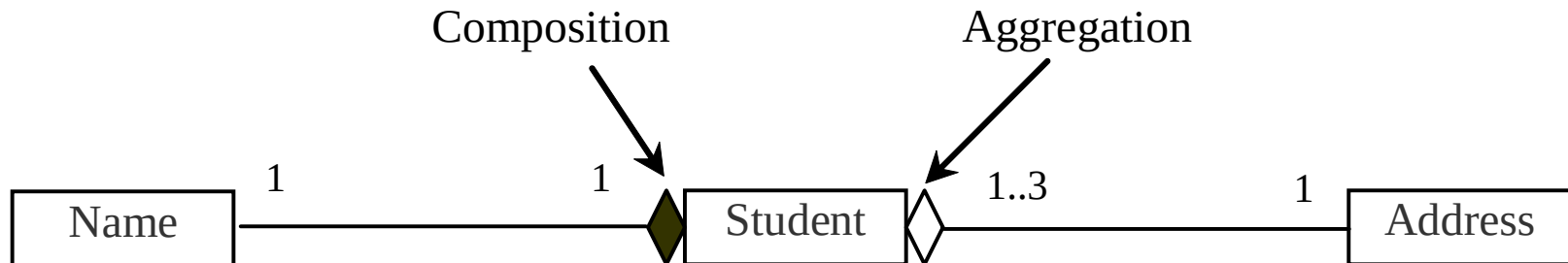
Aggregated class

```
public class Student {  
    private Name name;  
    private Address address;  
    ...  
}
```

Aggregating class

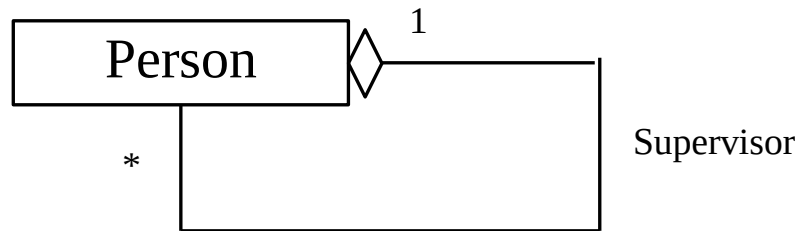
```
public class Address {  
    ...  
}
```

Aggregated class



Aggregation Between Same Class

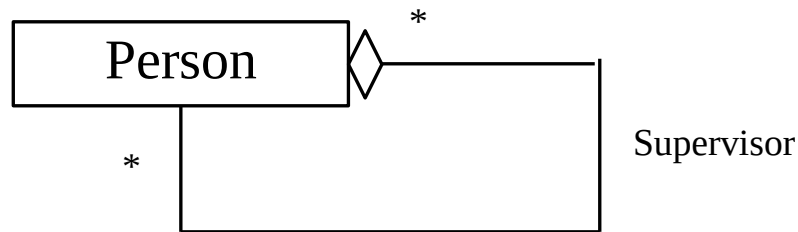
Aggregation may exist between objects of the same class.
For example, a person may have a supervisor.



```
public class Person {  
    // The type for the data is the class itself  
    private Person supervisor;  
    ...  
}
```

Aggregation Between Same Class

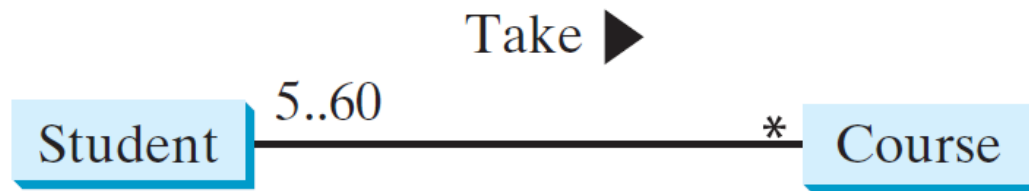
What happens if a person has several supervisors?



```
public class Person {  
    ...  
    private Person[] supervisors;  
}
```

Example: The Course Class

Suppose you need to process course information. Each course has a name and has students enrolled. You should be able to add/drop a student to/from the course.



Course

TestCourse

Example: The Course Class

Course
<pre>-courseName: String -students: String[] -numberOfStudents: int</pre>
<pre>+Course(courseName: String) +getCourseName(): String +addStudent(student: String): void +dropStudent(student: String): void +getStudents(): String[] +getNumberOfStudents(): int</pre>

The name of the course.

An array to store the students for the course.

The number of students (default: 0).

Creates a course with the specified name.

Returns the course name.

Adds a new student to the course.

Drops a student from the course.

Returns the students in the course.

Returns the number of students in the course.

Course

TestCourse

Wrapper Classes

- ❑ Boolean
- ❑ Character
- ❑ Short
- ❑ Byte
- ❑ Integer
- ❑ Long
- ❑ Float
- ❑ Double

NOTE:

- (1) The wrapper classes do not have no-arg constructors.
- (2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

The Integer and Double Classes

java.lang.Integer

-value: int

+MAX VALUE: int

+MIN VALUE: int

+Integer(value: int)

+Integer(s: String)

+byteValue(): byte

+shortValue(): short

+intValue(): int

+longVlaue(): long

+floatValue(): float

+doubleValue():double

+compareTo(o: Integer): int

+toString(): String

+valueOf(s: String): Integer

+valueOf(s: String, radix: int): Integer

+parseInt(s: String): int

+parseInt(s: String, radix: int): int

java.lang.Double

-value: double

+MAX VALUE: double

+MIN VALUE: double

+Double(value: double)

+Double(s: String)

+byteValue(): byte

+shortValue(): short

+intValue(): int

+longVlaue(): long

+floatValue(): float

+doubleValue():double

+compareTo(o: Double): int

+toString(): String

+valueOf(s: String): Double

+valueOf(s: String, radix: int): Double

+parseDouble(s: String): double

+parseDouble(s: String, radix: int): double

Numeric Wrapper Class Constructors

You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value. The constructors for Integer and Double are:

```
public Integer(int value)
```

```
public Integer(String s)
```

```
public Double(double value)
```

```
public Double(String s)
```

Numeric Wrapper Class Constants

Each numerical wrapper class has the constants MAX_VALUE and MIN_VALUE. MAX_VALUE represents the maximum value of the corresponding primitive data type. For Byte, Short, Integer, and Long, MIN_VALUE represents the minimum byte, short, int, and long values. For Float and Double, MIN_VALUE represents the minimum *positive* float and double values. The following statements display the maximum integer (2,147,483,647), the minimum positive float (1.4E-45), and the maximum double floating-point number (1.79769313486231570e+308d).

Conversion Methods

Each numeric wrapper class implements the abstract methods doubleValue, floatValue, intValue, longValue, and shortValue, which are defined in the Number class. These methods “convert” objects into primitive type values.

The Static valueOf Methods

The numeric wrapper classes have a useful class method, `valueOf(String s)`. This method creates a new object initialized to the value represented by the specified string. For example:

```
Double doubleObject = Double.valueOf("12.4");
```

```
Integer integerObject = Integer.valueOf("12");
```

The Static valueOf Methods

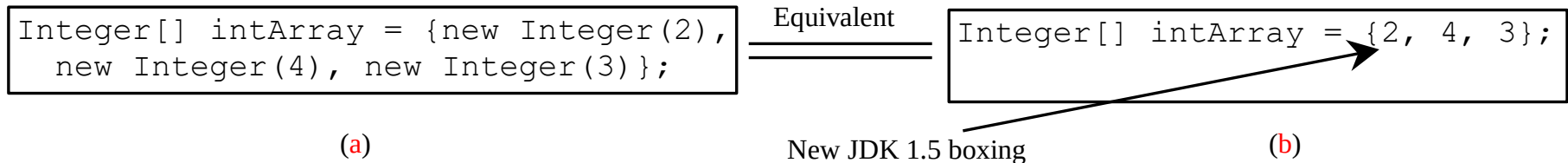
The constructors in the wrapper classes are deprecated in Java 9. It is preferred to use the static `valueOf` method to create an instance. Java enables frequently used wrapper objects to be reused through the `valueOf` method. An instance created using `valueOf` maybe shared, which is fine because the wrapper objects are immutable.

The Methods for Parsing Strings into Numbers

You have used the `parseInt` method in the `Integer` class to parse a numeric string into an `int` value and the `parseDouble` method in the `Double` class to parse a numeric string into a `double` value. Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value.

Automatic Conversion Between Primitive Types and Wrapper Class Types

JDK 1.5 allows primitive type and wrapper classes to be converted automatically. For example, the following statement in (a) can be simplified as in (b):



Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);

Unboxing

StringBuilder and StringBuffer

The `StringBuilder/StringBuffer` class is an alternative to the `String` class. In general, a `StringBuilder/StringBuffer` can be used wherever a string is used.

You can add, insert, or append new contents into a string buffer, whereas the value of a `String` object is fixed once the string is created.

StringBuilder/StringBuffer is more flexible than String, but String is more efficient.

StringBuilder Constructors

java.lang.StringBuilder
+StringBuilder()
+StringBuilder(capacity: int)
+StringBuilder(s: String)

Constructs an empty string builder with capacity 16.

Constructs a string builder with the specified capacity.

Constructs a string builder with the specified string.

Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: <i>aPrimitiveType</i>): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: <i>aPrimitiveType</i>): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Modifying Strings in the Builder

java.lang.StringBuilder

All these modification methods except **setCharAt** do two things:

1. *Change the contents of the string builder*
2. *Return the reference of the string builder*

For example: *StringBuilder stringBuilder1=stringBuilder.reverse();*

reverses the string in the builder and assigns the builder's reference to **stringBuilder1**. Thus, **stringBuilder** and **stringBuilder1** both point to the same **StringBuilder** object.

Returning the reference enables the **StringBuilder** methods to be invoked in a chain such as the following:

stringBuilder.reverse().delete(8, 11).replace(11, 15, "HTML");

The toString, capacity, length, setLength, and charAt Methods

java.lang.StringBuilder

+toString(): String

+capacity(): int

+charAt(index: int): char

+length(): int

+setLength(newLength: int): void

+substring(startIndex: int): String

+substring(startIndex: int, endIndex: int):
String

+trimToSize(): void

Returns a string object from the string builder.

Returns the capacity of this string builder.

Returns the character at the specified index.

Returns the number of characters in this builder.

Sets a new length in this builder.

Returns a substring starting at startIndex.

Returns a substring from startIndex to endIndex-1.

Reduces the storage size used for the string builder.

BigInteger and BigDecimal

If you need to compute with very large integers or high precision floating-point values, you can use the BigInteger and BigDecimal classes in the java.math package.

Both are *immutable*.

Both extend the Number class and implement the Comparable interface.

BigInteger and BigDecimal

```
BigInteger a = new BigInteger("9223372036854775807");  
BigInteger b = new BigInteger("2");  
BigInteger c = a.multiply(b); // 9223372036854775807 * 2  
System.out.println(c);
```

LargeFactorial

```
BigDecimal a = new BigDecimal(1.0);  
BigDecimal b = new BigDecimal(3);  
BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);  
System.out.println(c);
```

Thank you
Thank you

